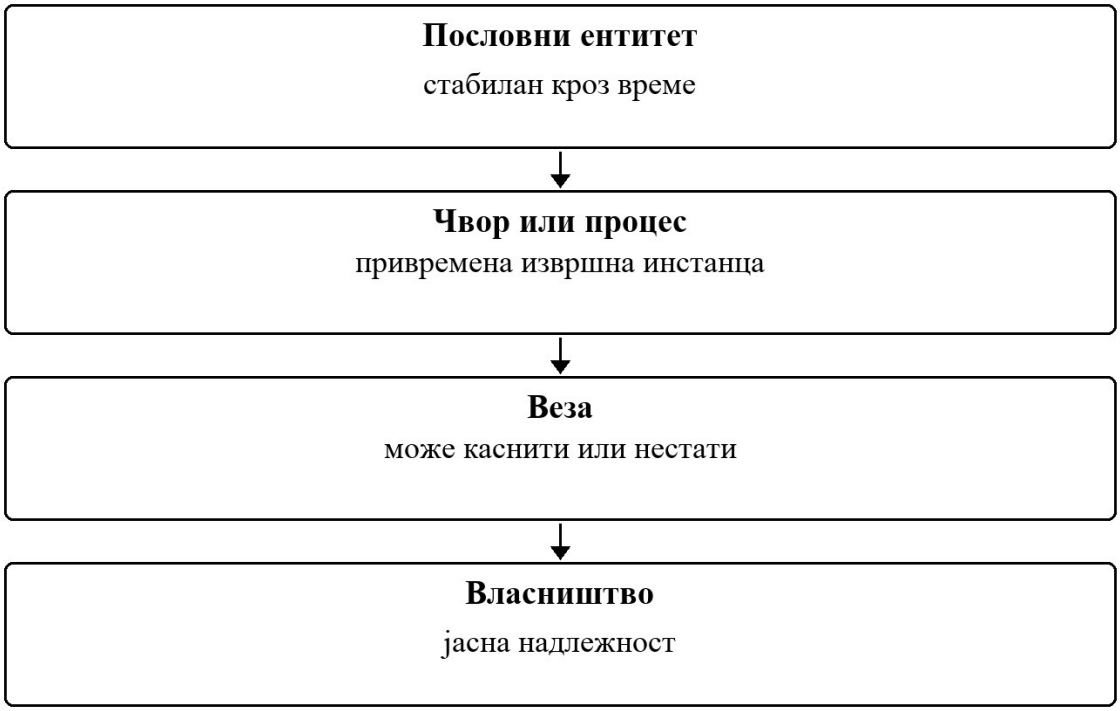
# Начин размишљања у дистрибуираном систему

Основе дистрибуираног програмирања · 01

Дистрибуирани систем није обична апликација распоређена на више процеса. Компоненте посматрају свет из различитих тренутака, мрежа може каснити или прекинути везу, а потврда операције може стићи након што је локални контекст већ промењен. Зато дизајн почиње јасним власништвом и границама одговорности.

Када се зна која компонента је надлежна за податак или операцију, лакше је објаснити шта се дешава при прекиду везе и ко доноси коначну одлуку. То спречава да више инстанци тихо раде исту ствар или да се нестанак процеса погрешно протумачи као нестанак пословног ентитета.




*Слика 1. Власништво, границе и неизвесност пре прве мрежне поруке*

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

**Фокус вежбе**

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

## 1.1 Систем не види све у исто време

- Процес, мрежна веза и пословни ентитет нису иста ствар.
- Истек времена значи да одговор није стигао на време, не да се операција сигурно није десила.
- Статус треба да изрази оно што систем зна, а не оно што претпоставља.

**Питање за дискусију**

Који податак или операција мора имати јасан ауторитет и како систем препознаје да је тај ауторитет привремено недоступан?

## 1.2 Како се ова тема види у развоју

Тим најпре описује границе система, одговорност сваке компоненте и стање које је за њу ауторитативно. Такав опис касније одређује где се раде провере, где се чува историја и шта се тестира када чвор привремено изгуби везу.

Честе грешке које треба препознати

- Изједначавање прекида везе са трајним кваром.
- Две инстанце које без правила мењају исто стање.
- Пословно правило скривено у комуникационом детаљу.

**Веза са остатком градива**

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

## 1.3 Пример: прекид везе није нестанак стања

Централна компонента престаје да прима хеартбеат од удаљеног чвора. Она зна да потврда није стигла, али не зна да ли је процес престао, мрежа прекинута или је порука изгубљена. Разлика је важна јер погрешна претпоставка може покренути дупли рад или погрешно затворити пословни ток.

- Статус треба да изрази последње поуздано сазнање система.
- Поновно повезивање не сме аутоматски значити да је настала нова пословна инстанца.
- Промена власништва над радом мора имати правило које спречава стари чвор да настави са радом.

**Како се сценарио анализира**

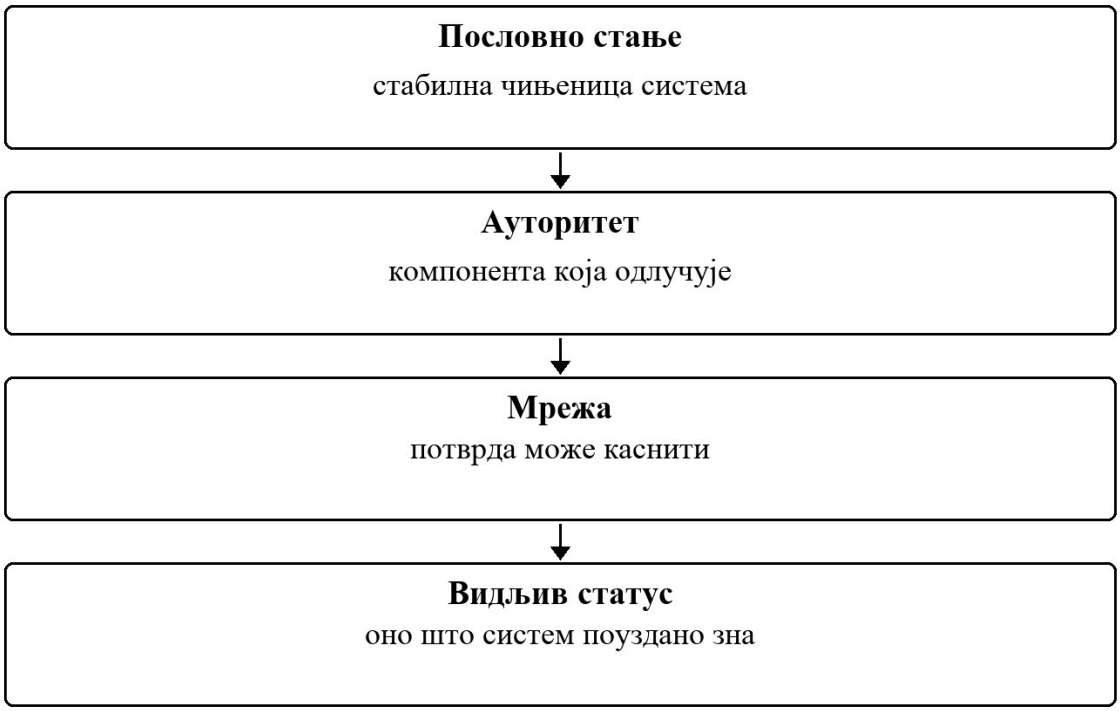
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

## 1.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

## 1.5 Шема ауторитета и неизвесности




*Слика 2. Шема ауторитета и неизвесности*

Процес и веза могу нестати, али пословно стање захтева јасан и стабилан ауторитет.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

## 1.6 Упоредни преглед елемената тока

*Табела 1. Елемент и Улога у току*

| Елемент | Улога у току | Питање које треба поставити |
|---|---|---|
| Пословни ентитет | стабилан кроз време | Шта се дешава када пословни ентитет није доступан или даје застарелу информацију? |
| Чвор или процес | привремена извршна инстанца | Шта се дешава када чвор или процес није доступан или даје застарелу информацију? |
| Веза | може каснити или нестати | Шта се дешава када веза није доступан или даје застарелу информацију? |
| Власништво | јасна надлежност | Шта се дешава када власништво није доступан или даје застарелу информацију? |

**Поређење појмова**

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

## 1.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Процес, мрежна веза и пословни ентитет нису иста ствар.

2. Истек времена значи да одговор није стигао на време, не да се операција сигурно није десила.

3. Статус треба да изрази оно што систем зна, а не оно што претпоставља.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

**Питање за вежбу**

Који податак или операција мора имати јасан ауторитет и како систем препознаје да је тај ауторитет привремено недоступан?

## 1.8 Извршиви пример

Пример раздваја пословни ентитет од процеса који га представља. Када удаљени чвор престане да се јавља, систем зна само колико дуго ћути; не зна да ли је процес пао, мрежа прекинута или је порука изгубљена. Зато статус говори оно што систем зна, а чвор остаје регистрован.

*Пример 1. Покретање демонстрације и тестова*

```
cd examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost
dotnet run --project src/Odp.Vezba01.ConsoleUi
dotnet test Odp.Vezba01.sln
```

Правило у коду

*Пример 2. examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost/src/Odp.Vezba01.Domain/Liveness/NodeLivenessPolicy.cs*

```
public NodeLivenessAssessment Evaluate(StationNode node, DateTimeOffset now)
{
    ArgumentNullException.ThrowIfNull(node);

    var silence = SilenceOf(node, now);
    if (silence < _thresholds.SuspectAfter)
        return new(NodeStatus.Online, NodeLivenessCodes.HeartbeatFresh, silence);
    if (silence < _thresholds.UnreachableAfter)
        return new(NodeStatus.Suspected, NodeLivenessCodes.HeartbeatLate, silence);
    return new(NodeStatus.Unreachable, NodeLivenessCodes.HeartbeatMissing, silence);
}

private static TimeSpan SilenceOf(StationNode node, DateTimeOffset now) =>
    now > node.LastHeartbeatAt ? now - node.LastHeartbeatAt : TimeSpan.Zero;
```

Тест који задаје квар

*Пример 3. examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost/tests/Odp.Vezba01.Tests/Application/MonitorNodesHandlerTests.cs*

```
[Test]
public void Assess_WhenNodeStaysSilent_ReportsItUnreachableButKeepsItRegistered()
{
    var clock = new ManualClock(Start);
    var registry = new InMemoryNodeRegistry([new StationNode("node-ns-1", "GS-NOVI-SAD",
Start)]);
    var handler = new MonitorNodesHandler(
        registry,
        new NodeLivenessPolicy(new LivenessThresholds(TimeSpan.FromSeconds(30),
TimeSpan.FromSeconds(90))),
        clock);

    clock.Advance(TimeSpan.FromMinutes(10));
    var view = handler.Assess().Single();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(view.Liveness.Status, Is.EqualTo(NodeStatus.Unreachable));
        Assert.That(view.Liveness.Code, Is.EqualTo(NodeLivenessCodes.HeartbeatMissing));
        Assert.That(registry.Find("node-ns-1"), Is.Not.Null);
    }
}
```

Испис демонстрације

*Пример 4. дотнет рун --пројецт src/Odp.Vezba01.ConsoleUi*

```
POČETAK [09:00:00]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
  heartbeat node-ns-1 poslat u 09:00:45 -> heartbeat_accepted
POSLE 45 s (javio se samo node-ns-1) [09:00:45]
  node-bg-1 (GS-BEOGRAD): Suspected | heartbeat_late | tišina 45 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
POSLE 105 s [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-ns-1 poslat u 09:00:20 -> heartbeat_out_of_order
STIGAO ZAKASNELI HEARTBEAT ZA node-ns-1 [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-bg-1 poslat u 09:01:45 -> heartbeat_accepted
node-bg-1 SE PONOVO JAVIO [09:01:45]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
```

Шта треба уочити

- Статус зависи само од дужине тишине; разлог тишине систем не зна и не претпоставља.
- Чвор који је `Unreachable` остаје у регистру: нестанак процеса није нестанак ентитета.
- Закаснели хеартбеат добија код `heartbeat_out_of_order` и не враћа познато стање уназад.

**Задатак уз пример**

Додати статус `Draining` за чвор који је најавио планирано гашење. Такав чвор не сме постати `Unreachable` док траје најављени период. Правило додати у `NodeLivenessPolicy`, увести нов код и покрити га тестом.

**Рад на вежби — примена на пројекат**

На додељеној пројектној целини применити принцип из теме „Начин размишљања у дистрибуираном систему". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.